

# Actividad práctica de JavaScript

*Clase 2 | Trabajo en grupos | Duración total 60 minutos*

Practiquen funciones, objetos y clases, almacenamiento en el navegador, timers y manipulación del DOM con eventos. Resuelvan los ejercicios con los ejemplos del material de la clase y comprueben cada resultado.

## Cómo trabajar

- Acuerden quién escribe y quién verifica; alternen estas tareas durante la actividad. Todas las personas del grupo deben poder explicar el código.
- Usen la consola del navegador para los ejercicios 1 a 4. Para el ejercicio 5, utilicen el HTML base incluido y escriban el JavaScript dentro de la etiqueta script.
- Guarden el código de cada ejercicio y una breve respuesta a su pregunta final. Los ejercicios son independientes.

Distribución del tiempo: ejercicio 1, 10 min; ejercicio 2, 12 min; ejercicio 3, 10 min; ejercicio 4, 10 min; ejercicio 5, 13 min; revisión final, 5 min.

## Ejercicio 1 Funciones de tres formas

*Tiempo 10 minutos | Consulten las páginas 3 a 8 del PDF*

Creen una función declarativa llamada `calcularArea` que reciba ancho y alto, y devuelva su producto. Asignen a alto el valor por defecto 1.

- Escriban la misma operación como función expresada llamada `areaExpresada` y como arrow function llamada `areaFlecha`. Conserven el parámetro por defecto en las tres.
- Prueben cada función con los argumentos (5, 3) y luego con (5). Muestren los resultados con `console.log`.

Comprueben: las tres versiones deben devolver 15 y 5, respectivamente. Cada función devuelve un valor; la impresión se realiza al invocarla.

Respondan: ¿cuál de estas formas permite llamar a la función antes de su declaración? Expliquen con el concepto de hoisting.

## Ejercicio 2 Una clase para un estudiante

*Tiempo 12 minutos | Consulten las páginas 13 a 20 del PDF*

Creen la clase Estudiante con un constructor que reciba nombre, edad y notas. El método promedio debe devolver la suma de las notas dividida por su cantidad, usando reduce y length. Trabajen con las notas dadas, que no están vacías.

- Creen una instancia con nombre "Ana", edad 20 y notas [6, 8, 10]. Impriman el resultado de promedio().
- Usen desestructuración para obtener el nombre e imprimirlo.
- Con spread, creen un objeto que copie los datos de la instancia y tenga edad 21. Impriman la edad de la instancia original y la de la copia.

Comprueben: promedio 8; nombre Ana; edades 20 y 21. La copia creada con spread contiene datos; para calcular el promedio usen la instancia original.

Respondan: ¿qué diferencia hay entre una propiedad y un método? Señalen un ejemplo de cada uno en su clase.

## Ejercicio 3 Datos persistentes y temporales

*Tiempo 10 minutos | Consulten las páginas 22 a 26 del PDF*

Trabajen en una página de práctica del navegador. Usen las claves indicadas para distinguir estos datos de otros que pudiera tener esa página.

- Creen el objeto { nombre: "Ana", edad: 20 }. Guárdenlo en localStorage con la clave "clase2\_estudiante", usando JSON.stringify.
- Recuperen el dato con getItem y JSON.parse. Impriman la propiedad nombre del objeto recuperado.
- Guarden "azul" en sessionStorage con la clave "clase2\_color" y lean su valor.
- Recarguen la misma pestaña y vuelvan a leer ambas claves. Al terminar, eliminen solo esas claves con removeItem y comprueben con getItem.

Comprueben: después de recargar, el nombre recuperado es Ana y el color es azul. Después de eliminar las claves, ambos getItem devuelven null.

Respondan: ¿cuál de los dos tipos de almacenamiento conserva los datos al cerrar el navegador? ¿Por qué el objeto se convirtió a texto antes de guardarlo?

## Ejercicio 4 Repetición controlada

*Tiempo 10 minutos | Consulten las páginas 27 a 29 del PDF*

Creen un contador que comience en 0. Usen `setInterval` para incrementarlo y mostrarlo cada 1000 milisegundos. Al llegar a 3, detengan el intervalo con `clearInterval` y muestren "Intervalo detenido". Usen el ejemplo de la página 29 como guía.

Agreguen un `setTimeout` independiente que muestre "Pasaron 2 segundos" después de 2000 milisegundos. Prueben una sola ejecución a la vez.

Comprueben: aparecen los valores 1, 2 y 3, sin llegar a 4; el mensaje del timeout aparece una sola vez. No exijan un orden específico entre mensajes programados para el mismo momento.

Respondan: ¿qué diferencia hay entre `setTimeout` y `setInterval`?

## Ejercicio 5 Un formulario que responde

*Tiempo 13 minutos | Consulten las páginas 33 a 36 y 44 a 50 del PDF*

Usen este HTML base. Agreguen su JavaScript dentro de `script`; la estructura ya está preparada para que se concentren en los eventos.

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Clase 2</title></head>
<body>
  <h1 id="titulo">Práctica del grupo</h1>
  <form id="formulario">
    <input id="nombre" type="text" placeholder="Nombre">
    <button type="submit">Enviar</button>
  </form>
  <p id="saludo">Escriban un nombre</p>
  <ul id="lista"></ul>
  <script>
    // Escriban aquí su JavaScript.
  </script>
</body>
</html>
```

- Seleccionen formulario, nombre, saludo y lista con `getElementById`. Al evento `input` del campo, actualicen el saludo con `textContent` y `event.target.value`.
- Al evento `submit`, usen `preventDefault`. Creen un `li` con el nombre del campo y agréguelo a la lista con `appendChild`.

Comprueben: al escribir Ana, el saludo muestra "Hola, Ana". Al enviar, aparece Ana en la lista y la página no se recarga. Repitan con Luis: deben quedar ambos nombres.

## Revisión final

Tiempo 5 minutos. Revisen juntos los resultados, corrijan errores y preparen el código de los cinco ejercicios con sus respuestas breves. Cada integrante debe poder explicar una parte de la solución.